

DMATH-ALGO

Elementare Datentypen

Version 2024

Serie 1

Lernziele

- Sie können mit den Objekttypen `boolean`, `integer`, `float`, `set`, `list` und `matrix` rechnen.
- Sie können `integer` und `float` in Binärdarstellung umrechnen und `float` Werte als Fließkommazahl nach IEEE 754 angeben.
- Sie verstehen den Begriff der Funktion.

In dieser Serie werden Konzepte, Terminologie und Notation im Zusammenhang mit den gebräuchlichsten Datentypen vorgestellt, die in den Vorlesungen DMATH-ALGO und DMATH-CODE und in der gesamten Informatik immer wieder vorkommen.

Diese grundlegenden Entitäten - *Boolesche Werte* (Wahr und Falsch), *Zahlen* (ganze Zahlen, rationale Zahlen und reelle Zahlen), *Mengen*, *Sequenzen* und *Funktionen* - sind auch die grundlegenden Datentypen, die wir in modernen Programmiersprachen verwenden. Praktisch jede interessante Anwendung in der Informatik nutzt diese grundlegenden Datentypen in grossem Umfang.

Frage 1: Was ist ein 'boolean'?

Die grundlegendste Einheit von Daten ist das **Bit**: eine einzelne Information, die genau zwei verschiedene Wert annehmen kann. Alle gespeicherten Daten in einem Digitalcomputer werden als eine Folge von Bits gespeichert. Die zwei Werte des Bits können aus verschiedenen Perspektiven betrachtet werden:

'1' und '0', 'Ein' und 'Aus', 'Ja' und 'Nein', 'Wahr' und 'Falsch'.

Bits, die unter der letztgenannten Perspektive betrachtet werden, haben einen besonderen Namen: **Booleans**.

Definition 1: Ein boolescher Wert ist entweder **True** oder **False**.

Boolesche Zahlen sind nach dem britischen Mathematiker George Boole (1815-1864) benannt, der als erster die Zahl 1 für True und 0 für False eingeführt hat.

Mit booleschen Werten können wir rechnen:

- Der einstellige Operator **not**:

$$\text{not } \text{F} = \text{T}$$

- Der zweistellige Operator **and**:

$$\text{T and T} = \text{T}$$

$$\text{T and F} = \text{F}$$

$$\text{F and T} = \text{F}$$

$$\text{F and F} = \text{F}$$

nur T, wenn beide Operanden T

- Der zweistellige Operator **or**:

$$\text{T or T} = \text{T}$$

$$\text{T or F} = \text{T}$$

$$\text{F or T} = \text{T}$$

$$\text{F or F} = \text{F}$$

nur F, wenn beide Operanden F

Aufgabe 1: Wenn in einer Rechnung keine Klammern stehen, wird in den meisten Programmiersprachen zuerst **not**, dann **and** und am Schluss **or** verarbeitet (**Priorität**). Gleiche Operatoren werden von links nach rechts verarbeitet (**Linksassoziativität**). Bestimmen Sie Schritt für Schritt den Wert der folgenden Berechnungen:

- a) False or True and False and True or False

$\text{F or } \text{F and T or F}$
 F or F or F
 F or F
 F

- b) False and True or False and not True or False

$\text{F and T or F and F or F}$
 F or F and F or F
 F or F or F
 F or F
 F

- c) not False or not not True and False or not True

$\text{T or not not T and F or not T}$
 $\text{T or not F and F or not T}$
 $\text{T or T and F or not T}$
 T or T and F or F
 T or F or F
 T or F
 T

not not ist
rechtsassoziativ (!)

Klammern in einer Rechnung werden immer zuerst ausgewertet. Bestimmen Sie Schritt für Schritt den Wert der folgenden Berechnungen:

- d) not(not True or not(False and not True)) or False

$\text{not(not T or not(F and F)) or F}$
 $\text{not(not T or not F) or F}$
 $\text{not(F or not F) or F}$
 not(F or T) or F
 not T or F
 F or F
 F

- e) not((False or not True) or True) and not(False or True)

$\text{not((F or F) or T) and not(F or T)}$
 $\text{not(F or T) and not(F or T)}$
 $\text{not T and not(F or T)}$
 not T and not T
 F and not T
 F and F
 F

Frage 2: Was ist ein 'integer'?

Die einfachsten Zahlen der Mathematik sind die ganzen Zahlen. Sie werden in der Informatik als **Integer** gespeichert.

Definition 2: Ein integer Wert ist entweder eine positive ganze Zahl, die Zahl Null oder eine negative ganze Zahl. Es sind also alle Zahlen

$$\mathbb{Z} = \{\dots, -4, -3, -2, -1, 0, 1, 2, 3, 4, \dots\}$$

zulässig.

Somit sind der einstellige Negationsoperator $-$ und die zweistelligen Operatoren Addition $+$, Subtraktion $-$ und Multiplikation $*$ für integer Werte wohldefiniert. Bei der Division gilt es eine Besonderheit zu beachten:

- Der zweistellige Operator $//$ (Ganzzahldivision):

$$10 // 3 = 3$$

$$10 // 7 = 1$$

$$10 // 5 = 2$$

- Der zweistellige Operator $\%$ (Rest):

$$10 \% 3 = 1$$

$$10 \% 7 = 3$$

$$10 \% 5 = 0$$

Alle integer Werte können mit Bits dargestellt werden:

$$\begin{aligned} 25 &= 12 \cdot 2 + 1 \quad \uparrow \\ 12 &= 6 \cdot 2 + 0 \quad \uparrow \\ 6 &= 3 \cdot 2 + 0 \quad \uparrow \\ 3 &= 1 \cdot 2 + 1 \quad \uparrow \\ 1 &= 0 \cdot 2 + 1 \quad \uparrow \text{ rückwärts einsetzen} \\ 3 &= (0 \cdot 2 + 1) \cdot 2 + 1 \\ &= 0 \cdot 2^2 + 1 \cdot 2 + 1 \\ 6 &= (0 \cdot 2^2 + 1 \cdot 2 + 1) \cdot 2 + 0 \\ &= 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2 + 0 \\ 12 &= (0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2 + 0) \cdot 2 + 0 \\ &= 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2 + 0 \\ 25 &= (0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2 + 0) \cdot 2 + 1 \\ &= 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2 + 1 \\ &= 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2 + 1 \end{aligned}$$

Damit haben wir die Binärdarstellung von der Zahl 25 gefunden:

$$(25)_2 = 11001$$

Aufgabe 2: Bestimmen Sie die Binärdarstellung von 409.

409	1
204	0
102	0
51	1
25	1
12	0
6	0
3	1
1	1
0	

also:

$$(409)_2 = 110011001$$

Frage 3: Was ist ein 'float'?

Eine beliebige Kommazahl gehört in der Mathematik zur Menge der reellen Zahlen $\mathbb{R}$. Alle diese Zahlen, bei denen sich die Nachkommastellen periodisch wiederholen, sind rational, können also als Brüche dargestellt werden. Die Menge der rationalen Zahlen wird mit $\mathbb{Q}$ bezeichnet.

Aufgabe 3: Bestimmen Sie einen Bruch, der $2.10\overline{365}$ darstellt:

$$\begin{array}{r} 100000 \cdot 2.10\overline{365} = 210365.\overline{365} \\ 100 \cdot 2.10\overline{365} = 210.\overline{365} \\ \hline \end{array}$$

$$99900 \cdot 2.10\overline{365} = 210155$$

$$\text{also: } 2.10\overline{365} = \frac{210155}{99900}$$

Alle reellen Zahlen können mit Bits dargestellt werden:

$$0.625 \cdot 2 = 1 + 0.25$$

$$0.25 \cdot 2 = 0 + 0.5$$

$$0.5 \cdot 2 = 1 + 0$$

$$0.5 = 1 \cdot 2^{-1} \quad \text{rückwärts einsetzen}$$

$$0.25 \cdot 2 = 0 + 1 \cdot 2^{-1}$$

$$0.25 = (0 + 1 \cdot 2^{-1}) \cdot 2^{-1}$$

$$= 0 \cdot 2^{-1} + 1 \cdot 2^{-2}$$

$$0.625 \cdot 2 = 1 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2}$$

$$0.625 = (1 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2}) \cdot 2^{-1}$$

$$= 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3}$$

Damit haben wir die Binärdarstellung von der Zahl 0.625 gefunden:

$$(0.625)_2 = 0.101$$

Das rückwärts Einsetzen ist für die Berechnung der Binärdarstellung nicht nötig. Die Bits können bereits vor dem Zurückrechnen abgelesen werden:

$$\begin{array}{rcll} 0.3 \cdot 2 & = & 0 + 0.6 & \\ 0.6 \cdot 2 & = & 1 + 0.2 & \leftarrow \\ 0.2 \cdot 2 & = & 0 + 0.4 & \\ 0.4 \cdot 2 & = & 0 + 0.8 & \\ 0.8 \cdot 2 & = & 1 + 0.6 & \leftarrow \\ 0.6 \cdot 2 & = & 1 + 0.2 & \text{periodische Wiederholung} \\ \vdots & = & \vdots & \end{array}$$

Die Rechnung stoppt nicht, aber die periodische Wiederholung in der Binärdarstellung von 0.3 lässt sich ablesen:

$$(0.3)_2 = 0.0\overline{1001}$$

Aufgabe 4: Bestimmen Sie die Binärdarstellung von $1/6$.

$$\begin{array}{rcll} 1/6 \cdot 2 & = & 0 + 1/3 & \\ 1/3 \cdot 2 & = & 0 + 2/3 & \\ 2/3 \cdot 2 & = & 1 + 1/3 & \end{array}$$

$$\text{Also: } (1/6)_2 = 0.0\overline{01}$$

Die Nachkommastellen der Kreiszahl

$$\pi = 3.1415926535897932384626433832795028841971693993751058 \dots$$

haben zum Beispiel keine periodische Wiederholung. Solche irrationale Kommazahlen kann ein Rechner nicht exakt speichern, weil unendlich viel Speicherplatz für die Nachkommastellen gebraucht würde.

Aber auch die Nachkommastellen von rationalen Zahlen mit einer unendlichen periodischen Wiederholung (verschieden von Null) können nicht gespeichert werden. Kommazahlen mit vorgegebener Genauigkeit bei den Nachkommastellen werden als *float* bezeichnet.

Definition 3: Ein float Wert ist eine Kommazahl mit endlich vielen Nachkommastellen. Je nach Standard sind mehr oder weniger Werte für float möglich.

Ein gängiger Standard ist die Darstellung als **Flieszkommazahl** nach IEEE 754. Das Rechnen mit float Werten kann also unerwartete Ergebnisse haben. Zum Beispiel liefert in Python der Vergleich

$$0.7 + 0.1 \neq 0.9 - 0.1$$

den boolean False.

Aufgabe 5: Bestimmen Sie die Darstellung als Flieszkommazahl im 32-bit (float in Java) und im 64-bit (double in Java) Format von den folgenden Zahlen:

a) die Binärzahl $0.001\overline{0}$ $= 0.00101010\dots$
 $= 1.01010\dots \cdot 2^{-3}$

32-bit : $-3 + 127 = 124 \rightarrow (124)_2 = 1111100$

$$\begin{array}{c} 0 \quad 01111100 \quad 01010101\dots01011 \\ \hline \text{1 bit} \quad \leftarrow \text{8 bit} \quad \leftarrow \text{23 bit} \end{array}$$

64-bit : $-3 + 1023 = 1020 \rightarrow (1020)_2 = 111111100$

$$\begin{array}{c} 0 \quad 0111111100 \quad 010101\dots0101 \\ \hline \text{1 bit} \quad \leftarrow \text{11 bit} \quad \leftarrow \text{52 bit} \end{array}$$

b) die Dezimalzahl 35.2

35	1	$0.2 \cdot 2 = 0 + 0.4$
17	1	$0.4 \cdot 2 = 0 + 0.8$
8	0	$0.8 \cdot 2 = 1 + 0.6$
4	0	$0.6 \cdot 2 = 1 + 0.2$
2	0	
1	1	
0		

also: $(35.2)_2 = 100011.\overline{0011}$
 $= 1.000110011\dots \cdot 2^5$

32-bit : $5 + 127 = 132 \rightarrow (132)_2 = 10000100$

$$0 \quad 10000100 \quad 00011001100\dots1101$$

64-bit : $5 + 1023 = 1028 \rightarrow (1028)_2 = 10000000100$

$$0 \quad 10000000100 \quad 00011001100\dots11010$$

Frage 4: Was sind 'set', 'list' und 'matrix'?

Neben den primitiven Objekttypen `boolean`, `integer` und `float` gibt es auch verschiedene Sammlungen von Objekten, wie Listen und Arrays in Programmiersprachen.

Definition 4: Eine Menge (set) ist eine ungeordnete Sammlung von Objekten, ohne Rücksicht auf Wiederholung. Wenn ein Objekt x zu einer Menge M gehört, sagen wir x ist ein Element von M und schreiben $x \in M$. Wenn x nicht zur Menge M gehört, schreiben wir $x \notin M$.

Wir können die Elemente einer Menge aufzählen

$$A = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\} = \{3, 2, 8, 3, 1, 5, 7, 6, 4, 0\}$$

oder wir können sie beschreiben

$$B = \{x : x \text{ ist eine Ziffer}\}.$$

Mengen können wir vergleichen:

- Die zweistellige Relation $=$ (Gleichheit):

$$\begin{aligned} \{a, b, c\} &= \{b, c, a\} \\ \{a, b, c\} &\neq \{a, b, c, x\} \\ \{a, b, c\} &\neq \{a, b, x\} \end{aligned}$$

- Die zweistellige Relation $\subset$ (Teilmenge):

$$\begin{aligned} \{a, b, c\} &\subset \{a, b, c, x\} \\ \{a, b, c\} &\subset \{a, b, c\} \\ \{ \} &\subset \{a, b, c\} \\ \{a, b\} &\not\subset \{a, c, x\} \end{aligned}$$

Aufgabe 6: Bestimmen Sie alle Teilmengen von der Menge $\{a, b, c, d\}$:

$$\begin{array}{cccccc} \{ \} & \{a\} & \{a, b\} & \{a, b, c\} & \{a, b, c, d\} \\ & \{b\} & \{a, c\} & \{a, b, d\} & \\ & \{c\} & \{a, d\} & \{a, c, d\} & \\ & \{d\} & \{b, c\} & \{b, c, d\} & \\ & & \{b, d\} & & \\ & & \{c, d\} & & \end{array}$$

Mit Mengen können wir auch rechnen:

- Der zweistellige Operator $\cap$ (Durchschnitt):

$$\{a, b, c\} \cap \{b, c, d\} = \{b, c\}$$

$$\{a, b\} \cap \{c, d\} = \{\}$$

- Der zweistellige Operator $\cup$ (Vereinigung):

$$\{a, b, c\} \cup \{b, c, d\} = \{a, b, c, d\}$$

$$\{a, b\} \cup \{c, d\} = \{a, b, c, d\}$$

- Der zweistellige Operator $\setminus$ (Differenz):

$$\{a, b, c\} \setminus \{b, c, d\} = \{a\}$$

$$\{a, b\} \setminus \{c, d\} = \{a, b\}$$

- Der einstellige Operator $\overline{}$ (Komplement):

können wir für Teilmengen einer Menge anwenden!

wenn z.B. $\{a, b\} \subset \{a, b, c, d, e\}$

dann ist $\overline{\{a, b\}} = \{c, d, e\}$

Definition 5: Eine Liste (list) ist eine geordnete Sammlung von Objekten, mit Rücksicht auf Wiederholung. Die Objekte einer Liste heissen Komponenten oder Einträge. Weil die Objekte geordnet sind, können sie von der ersten Komponente bis zur letzten Komponente nummeriert werden.

Liste werden in der Mathematik mit runden Klammern notiert, zum Beispiel:

$$a = (5, 3, 8, 8, 1)$$

Mit a_i bezeichnen wir die Komponente am i -ten Platz in der Liste a . In vielen Programmiersprachen wird gleichbedeutend die *Slicing-Syntax* $a[i]$ verwendet:

$$a_2 = a[2] = 3 \quad (\text{wenn wir mit Index } i = 1 \text{ starten})$$

Wenn mit den Komponenten zweier gleich langer Listen a und b gerechnet werden kann, dann können wir auch die Listen mit denselben Operationen *komponentenweise* verrechnen:

- $(1, 3, -1, 0, 1.5) + (1, 2, 2, 1, 0.5) = (2, 5, 1, 1, 2)$
- $(\text{True}, \text{True}, \text{False}) \text{ and } (\text{False}, \text{True}, \text{False}) = (\text{F}, \text{T}, \text{F})$

Genau so können wir auch einzelne kompatible Werte mit einer Liste verrechnen, indem wir den einzelnen Wert zu einer Liste gleicher Grösse erweitern (*Broadcasting*) und dann wieder komponentenweise rechnen:

- $3 \cdot (2, 5, 4, -2) = (3, 3, 3, 3) \cdot (2, 5, 4, -2)$
 $= (6, 15, 12, -6)$
- $3 - (0, 1, 2, 3, 2, 1, 0) = (3, 3, 3, 3, 3, 3, 3) - (0, 1, 2, 3, 2, 1, 0)$
 $= (3, 2, 1, 0, 1, 2, 3)$
- $\text{not}(\text{True}, \text{False}, \text{False}, \text{True}, \text{False}) = (\text{F}, \text{T}, \text{T}, \text{F}, \text{T})$

↑
1 stelliger Operator wirkt auf alle Komponenten

Definition 6: Eine Matrix (matrix) ist eine zweidimensionale, rechteckig geordnete Sammlung von Objekten, mit Rücksicht auf Wiederholung. Die Objekte einer Matrix heissen Komponenten oder Einträge. Weil die Objekte geordnet sind, können sie zeilen- und spaltenweise nummeriert werden.

Matrizen werden in der Mathematik mit runden Klammern notiert, zum Beispiel:

$$A = \begin{pmatrix} 5 & 3 & 8 & 8 & 1 \\ 1 & 0 & 0 & 9 & 7 \end{pmatrix}$$

Die Matrix A hat zwei Zeilen und fünf Spalten, ihre *Dimension* wird mit 2×5 angegeben. So hat also eine $m \times n$ Matrix m Zeilen und n Spalten. Mit a_{ij} bezeichnen wir die Komponente in der i -ten Zeile und j -ten Spalte. In vielen Programmiersprachen wird gleichbedeutend die *Slicing-Syntax* $A[i, j]$ oder $A[i][j]$ verwendet:

$$a_{24} = A[2, 4] = A[2][4] = 9 \quad (\text{wenn wir mit } i = 1 \text{ und } j = 1 \text{ starten})$$

Matrizen gleicher Dimension werden auch komponentenweise verrechnet.

$$\circ \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 7 & 5 \end{pmatrix} + \begin{pmatrix} 0 & 4 \\ 1 & 1 \\ 5 & 2 \end{pmatrix} = \begin{pmatrix} 1+0 & 2+4 \\ 3+1 & 4+1 \\ 7+5 & 5+2 \end{pmatrix} = \begin{pmatrix} 1 & 6 \\ 4 & 5 \\ 12 & 7 \end{pmatrix}$$

$$\circ \begin{pmatrix} \text{False} & \text{True} \\ \text{False} & \text{True} \end{pmatrix} \text{ or } \begin{pmatrix} \text{True} & \text{False} \\ \text{False} & \text{True} \end{pmatrix} = \begin{pmatrix} \text{True or True} & \text{True or False} \\ \text{True or False} & \text{True or True} \end{pmatrix} = \begin{pmatrix} \text{True} & \text{True} \\ \text{True} & \text{True} \end{pmatrix}$$

Das Broadcasting funktioniert auch, wenn eine $m \times n$ Matrix mit einer 1×1 oder $m \times 1$ oder $1 \times n$ Matrix verrechnet wird:

$$\begin{aligned} \circ & \overset{1 \times 1}{2} + \overset{3 \times 4}{\begin{pmatrix} 1 & 2 & 1 & 1 \\ 3 & 4 & 2 & 2 \\ 3 & 5 & 1 & 1 \end{pmatrix}} - \overset{3 \times 1}{\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}} + \overset{1 \times 4}{(0 \ 1 \ 1 \ 2)} \cdot \overset{3 \times 4}{\begin{pmatrix} 1 & 2 & 3 & 2 \\ 1 & 5 & 3 & 3 \\ 1 & 2 & 2 & 5 \end{pmatrix}} = \\ & \begin{pmatrix} 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \end{pmatrix} + \begin{pmatrix} 1 & 2 & 1 & 1 \\ 3 & 4 & 2 & 2 \\ 3 & 5 & 1 & 1 \end{pmatrix} - \begin{pmatrix} 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 3 & 3 & 3 \end{pmatrix} + \begin{pmatrix} 0 & 1 & 1 & 2 \\ 0 & 1 & 1 & 2 \\ 0 & 1 & 1 & 2 \end{pmatrix} \cdot \begin{pmatrix} 1 & 2 & 3 & 2 \\ 1 & 5 & 3 & 3 \\ 1 & 2 & 2 & 5 \end{pmatrix} \\ & = \begin{pmatrix} 2 & 5 & 5 & 6 \\ 3 & 3 & 5 & 8 \\ 2 & 6 & 2 & 10 \end{pmatrix} \end{aligned}$$

Frage 5: Was ist eine 'function'?

Definition 7: Sei D eine Menge von Input-Werten (Definitionsbereich) und W eine Menge von Output-Werten (Wertevorrat). Eine Funktion mit Namen f (function) ordnet jedem Element in D genau ein Element in W zu.

Diese eindeutige Zuordnung von Input zu Output wird mit der Pfeilnotation

$$f : D \longrightarrow W$$

bezeichnet. Wenn zum Input x der Output y zugeordnet wird, schreiben wir

$$x \mapsto y \quad \text{oder auch} \quad f(x) = y.$$

Eine präzise, schrittweise Angabe, die für jeden Input $x \in D$ beschreibt, wie der zugehörige Output $y = f(x)$ bestimmt wird, heisst **Algorithmus**.

Sei zum Beispiel $D = \mathbb{N}$ die Menge aller natürlicher Zahlen und W die Menge aller endlich langen Zeichenketten aus 0 und 1. Dann könnten wir die Funktion

$$\text{decToBin} : D \longrightarrow W$$

definieren, die jeder natürlichen Zahl x ihre Binärdarstellung $(x)_2$ zuordnet. Einen Algorithmus für die Berechnung haben wir schon kennengelernt (hier in Python):

```
def decToBin(x):
    y = ''
    temp = x
    while temp != 0:
        if temp % 2 == 0:
            y = '0' + y
        else:
            y = '1' + y
        temp = temp // 2
    return y
```

Aufgabe 7 (optional):

- a) Implementieren Sie eine Funktion, die für eine rationale Kommazahl die Bruchdarstellung zurückgibt.
- b) Implementieren Sie eine Funktion, die für rationale Zahlen die Binärdarstellung als Zeichenkette zurückgibt.
- c) Implementieren Sie eine Funktion, die für eine Kommazahl die Quadratwurzel in Binärdarstellung auf eine vorgegebene Anzahl Ziffern nach dem Komma zurückgibt.